Executive Master Intelligence artificielle & science des données 2025-2026 Promo 7

Cours : Traitement distribué de données massive sur Hadoop, Spark et Kafka

Project CCF (Finding Connected Component in Graphs)

I. Table des matières

II. Auteurs 2
III. Rappel consigne du projet 2
IV. Introduction 2
V. Matériel et Méthode 2
A. Jeu de données (dataset) 2
1. Google 3
2. NotreDame 3
B. Environnement de travail 3
1. Machine 1 – 8Go RAM 4 coeurs 3
2. Machine 2 – 16Go RAM CPU 8 cœurs, GPU 8 cœurs (configuration 2) 3
C. Pipeline 5
VI. Description de l'algorithme 6
VII. Résultats 7
A. Représentation du traitement par l'algorithme CCF 7
B. Représentation du jeu de donnée Google 8
C. Traitement en Python 9
1. Machine 1 sans paramètres Spark : local[*] 9
2. Effet de la parallélisation 10
3. Machine 2 11
4. Comparaison machine et pipeline 12
5. Conclusion Python 13
D. Scala 13
E. Comparaison python versus scala 15
VIII. Conclusion 15

II. Auteurs

Laure BOIZEAU : laure.boizeau@gmail.com

Xuan PENG : xuan.peng@outlook.fr

III. Rappel consigne du projet

Recherche de composants connexes dans un graphe selon l'algorithme décrit dans l'article

https://www.cse.unr.edu/~hkardes/pdfs/ccf.pdf

• Le travail consiste à comprendre l'algorithme MapReduce et à le coder dans Spark en utilisant RDD et DataFrames.

• Des implémentations Python et Scala doivent être fournies.

• Une analyse expérimentale comparant les versions RDD et DataFrame doit être réalisée sur des graphes de taille croissante.

• Pour les petits graphes, utilisez Databricks ; pour les plus grands (< 20 Go), utilisez le cluster GC.

IV. Introduction

Un graphe est une structure mathématique utilisée pour modéliser les relations deux à deux entre objets. Il est composé de sommets (également appelés nœuds (nodes) ou points) reliés par des arêtes (edges) (également appelées liens ou lignes).

De nombreux problèmes pratiques peuvent être représentés par des graphes : ils permettent de modéliser divers types de relations et de processus dans les systèmes physiques, biologiques, sociaux et d'information. Trouver des composantes connexes dans un graphe est un problème bien connu dans de nombreux domaines d'application. À cet effet, Hakan Kardes, Siddharth Agrawal, Xin Wang et Ang Sun ont publié « CCF : Calcul rapide et évolutif de composantes connexes dans MapReduce », ils proposent une méthode appelée **Connected Component Finder (CCF)**, conçue pour traiter des graphes massifs via le paradigme MapReduce et son implémentation distribuée dans Hadoop.

Cette approche se distingue par sa capacité à gérer des volumes extrêmement importants de données : les auteurs l'ont validée sur un graphe de près de 6 milliards de nœuds et 92 milliards d'arêtes, démontrant une montée en charge efficace là où d'autres algorithmes échouaient.

Dans ce projet, nous expliquons l'algorithme et les principaux concepts de CCF. Nous réalisons ensuite une implémentation en **PySpark** et en **Scala**, en utilisant successivement les **RDD** et les **DataFrames**. Enfin, nous analysons la scalabilité de ces solutions sur des jeux de données de taille croissante.

V. Matériel et Méthode

A. Jeu de données (dataset)

Pour la mise en pratique des concepts de traitement distribué et évaluer les performances, deux jeux de données issus de la collection Stanford Large Network Dataset (SNAP) ont été utilisés, avec des volumes d'information différents :

1. Google

https://snap.stanford.edu/data/web-Google.html

Name	Type	Nodes	Edges	Description	Collection date
web-Google	Directed	875k	5,1M	Web graph from Google	2002

2. NotreDame

Stanford Large Network Dataset Collection web site

Name	Type	Nodes	Edges	Description	Collection date
web-NotreDame	Directed	325k	1,5M	Web graph of Notre Dame	1999

B. Environnement de travail

Le projet a été réalisé dans deux environnements distincts, avec des configurations matérielles différentes.

1. Machine 1 – 8Go RAM 4 cœurs, GPU 2 cœurs

Travail effectué dans un conteneur **Docker** basé sur l'image **jupyter/all-spark-notebook**:x86_64-ubuntu-22.04, avec l'installation de **Toree** pour exécuter les notebooks Scala et PySpark. Au lancement du conteneur, les ressources ont été limitées à --memory="6g" et --cpus="3", afin de s'adapter aux contraintes matérielles de la machine hôte.

Dans le notebook, des configurations Spark spécifiques ont ensuite été appliquées :

a) Mode Spark Libre

Utilisation de tous les cœurs disponibles pour les tests (local[*])

```
spark = SparkSession.builder.appName("projet).master("local[*]").getOrCreate()
sc = SparkContext.getOrCreate(SparkConf().setMaster("local[*]"))
```

b) Mode Spark Fixé (configuration 1)

Limitation volontaire des ressources mémoire et CPU pour contrôler l'exécution des tests.

```
spark = SparkSession.builder.appName("projet") \
   .master("local[6]") \
   .config("spark.driver.memory", "1g") \
   .config("spark.executor.memory", "1g") \
   .config("spark.executor.cores", "3") \
   .config("spark.sql.shuffle.partitions", "4") \
   .getOrCreate()

sc = SparkContext.getOrCreate(SparkConf().setMaster("local[6]"))
```

2. Machine 2 – 16Go RAM CPU 8 cœurs, GPU 8 cœurs (configuration 2)

Les outils de développement utilisés sont Kaggle Notebook pour Python et IntelliJ IDEA pour Scala, ce dernier étant configuré selon les spécifications définies dans le fichier **build.sbt**.

Ajustement avancé des paramètres mémoire, partitions et tolérances réseau pour supporter des volumes de données plus importants.

```
spark = SparkSession.builder \
  .appName("ConnectedComponentsFinder") \
  .config("spark.driver.memory", "12g") \
  .config("spark.executor.memory", "12g") \
  .config("spark.memory.fraction", "0.6") \
  .config("spark.memory.storageFraction", "0.3") \
  .config("spark.sql.adaptive.enabled", "true") \
  .config("spark.sql.shuffle.partitions", "50") \
  .config("spark.sql.autoBroadcastJoinThreshold", "10485760") \
  .config("spark.memory.offHeap.enabled", "true") \
  .config("spark.memory.offHeap.size", "4g") \
  .config("spark.executor.memoryOverhead", "2g") \
  .config("spark.driver.memoryOverhead", "2g") \
  .config("spark.network.timeout", "1200s") \
  .config("spark.sql.broadcastTimeout", "1200") \
  .getOrCreate()

sc = spark.sparkContext
```

build.sbt (pour Scala):

```
ThisBuild / scalaVersion := "2.13.12"
ThisBuild / scalacOptions ++= Seq("-deprecation", "-feature")

libraryDependencies += "org.apache.spark" %% "spark-core" % "3.5.0"
libraryDependencies += "org.apache.spark" %% "spark-sql" % "3.5.0"

// Set Java compatibility
javacOptions ++= Seq("-source", "11", "-target", "11")
scalacOptions += "-target:11"

// Increase stack size for Scala compiler
javaOptions ++= Seq("-Xss4m")

// add to run and test
run / javaOptions ++= Seq("-Xss4m")
test / javaOptions ++= Seq("-Xss4m")
test / javaOptions ++= Seq("-Xmx2g")

run / javaOptions ++= Seq(
  "--add-opens=java.base/java.lang=ALL-UNNAMED",
  "--add-opens=java.base/java.nio=ALL-UNNAMED",
  "--add-opens=java.base/sun.nio.ch=ALL-UNNAMED",
  "--add-exports=java.base/sun.nio.ch=ALL-UNNAMED",
  "--add-opens=java.base/javax.security.auth=ALL-UNNAMED",
  "--add-opens=java.base/java.security=ALL-UNNAMED",
```

```
  "--add-opens=java.base/java.util=ALL-UNNAMED",
  "--add-opens=java.base/java.util.concurrent=ALL-UNNAMED",
  "--add-opens=java.base/java.util.concurrent.atomic=ALL-UNNAMED",
  "--add-opens=java.base/jdk.internal.ref=ALL-UNNAMED",
  "--add-opens=java.base/jdk.internal.reflect=ALL-UNNAMED",
  "--add-opens=java.base/jdk.internal.math=ALL-UNNAMED",
  "--add-opens=java.base/jdk.internal.util=ALL-UNNAMED",
  "--add-opens=java.base/jdk.internal.access=ALL-UNNAMED",
  "--add-opens=java.base/jdk.internal=ALL-UNNAMED",
  "--add-opens=java.security.jgss/sun.security.krb5=ALL-UNNAMED",
  "--enable-native-access=ALL-UNNAMED"
)

// add to test
test / javaOptions ++= Seq(
  "--add-opens=java.base/java.lang=ALL-UNNAMED",
  "--add-opens=java.base/javax.security.auth=ALL-UNNAMED",
  "--enable-native-access=ALL-UNNAMED"
)

// Reduce Spark memory usage for local development
run / javaOptions ++= Seq(
  "-Xmx8g",  // Limit Java heap to 4GB instead of default
  "-Xms6g",
  "-XX:MaxPermSize=512m",
  "-XX:MaxMetaspaceSize=2g", //1g
  "-XX:+UseG1GC",
  "-XX:ParallelGCThreads=4"
)
```

C. Pipeline

L'implémentation de l'algorithme CCF a été réalisée dans trois notebooks distincts :

Les pipelines 1 et 2 sont codés en Python, ils ont été développées par deux programmeurs différents.

- **Pipeline_1_python_spark.ipynb** (kernel Python)
    - Projet_Python_Comp.csv pour l'analyse du temps d'exécution sur différents volumes de données, en utilisant les traitements **RDD** et **DataFrame**.
- **Pipeline_2_python_spark.ipynb** (kernel Python)

Le pipeline 3 est codé en Scala.

- **Pipeline_3_scala_spark.ipynb** (kernel Scala)
    - Projet_Scala_Comp.csv : analyse du temps d'exécution selon différents algorithmes sur des volumes variables de données, avec traitement **RDD** et **DataFrame**.
    - Projet_scala_RDD_test_parall.csv : analyse du temps d'exécution en fonction du nombre de parallélisations, avec 100 000 entrées, en utilisant l'algorithme **CCF-Iterate+ CCF-dedup = standard**.

- Projet_nb_connect_by_group_5M.csv : récapitulatif du nombre de connexion existante pour chaque groupe après application de l'algorithme CCF

Agrégation des résultats obtenus sur le dataset Google, regroupant les tests réalisés sur différentes machines, configurations Spark, pipelines et variantes de l'algorithme, dans le fichier

- Synthese_test.csv : récapitulatif des temps d'exécution selon les tests

Enfin, un quatrième notebook a été utilisé pour l'analyse et la visualisation des résultats :

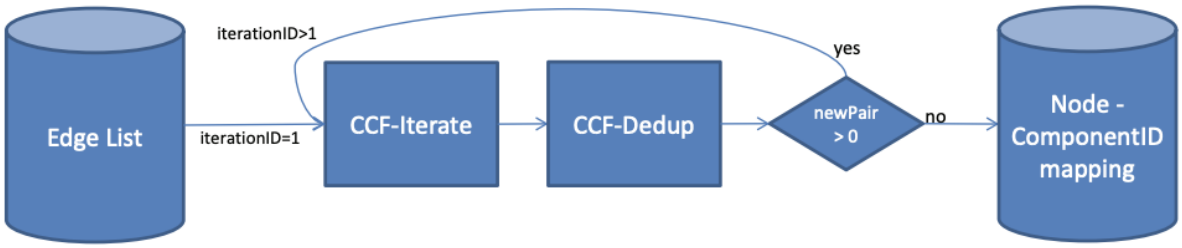
- **Projet_analyse.ipynb** (kernel Python) Génération des graphiques présentés dans ce rapport à partir des données issues des notebooks précédents.

VI. Description de l'algorithme

Voici le fonctionnement de CCF :

- Il prend en entrée une liste de toutes les arêtes.
- Il renvoie en sortie la correspondance (c'est-à-dire un tableau) de chaque nœud avec son identifiant de composant correspondant (c'est-à-dire le plus petit identifiant de nœud de chaque composant connexe auquel il appartient).

Pour ce faire, la chaîne d'opérations suivante est exécutée :



igure 1. Connected Component Finder(CCF) Module

Deux tâches s'exécutent de manière itérative jusqu'à ce qu'aucun nouveau pair connecté ne soit trouvé aux composants existants :

CCF-Iterate

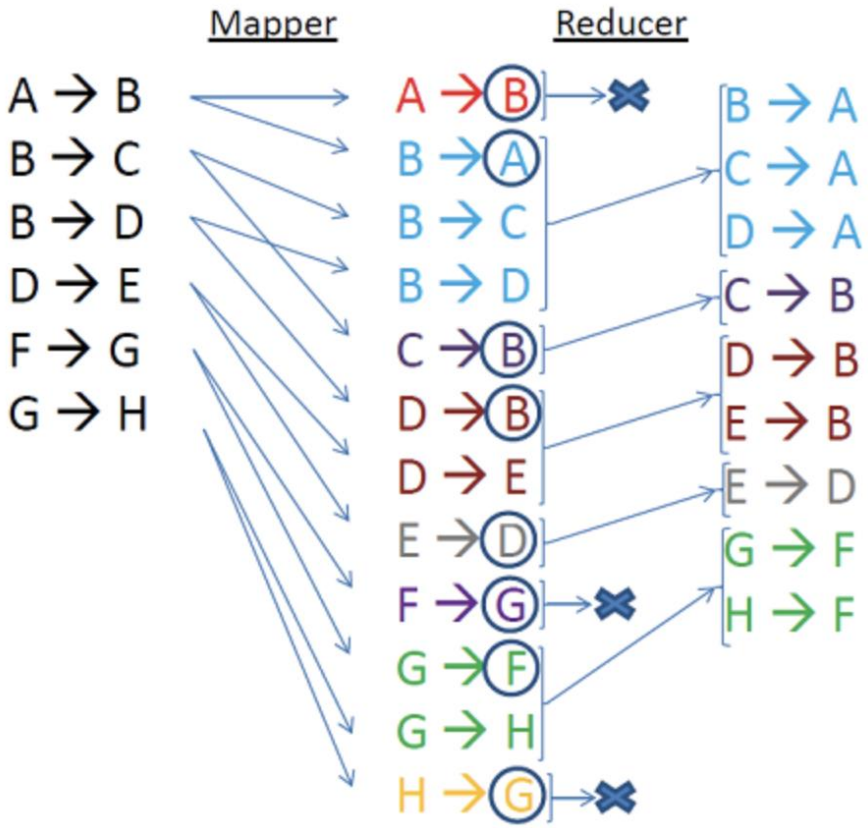
Cette tâche génère une liste d'adjacences AL = (a1, a2, ..., an) pour chaque nœud, c'est-à-dire la liste des nouveaux nœuds appartenant au même composant connecté. À chaque fois, l'identifiant du nœud est mis à jour lorsqu'un nouveau nœud pair apparaît dont l'identifiant devient le nouveau minimum.

S'il n'y a qu'un seul nœud dans AL, nous générerons la paire obtenue à l'itération précédente. En revanche, s'il y a plusieurs nœuds dans AL, le processus n'est pas encore terminé : une nouvelle itération est nécessaire pour trouver d'autres pairs.

CCF-Dedup

Lors de la tâche CCF-Iterate, la même paire peut être émise plusieurs fois. La deuxième tâche, CCF-Dedup, déduplique simplement la sortie de la tâche CCF-Iterate afin d'améliorer l'efficacité de l'algorithme.

Décomposons l'ensemble du processus pièce par pièce en utilisant l'exemple illustré ci-dessous :



- Pour chaque arête, le mappeur CCT-Iterate émet les paires (k, v) et (v, k) afin que *A* soit dans la liste d'adjacence de *B* et inversement.
- En phase de réduction, tous les nœuds adjacents sont regroupés, puis les paires sont triées par clés.
- Toutes les valeurs sont analysées groupe par groupe :
    - une première fois pour trouver la valeur minimale (minValue)
    - une seconde fois pour émettre un nouvel identifiant de composant si nécessaire.
- Le compteur global Nouvelle Paire est initialisé à 0. Si un nouveau nœud homologue est trouvé pour chaque composant, le compteur est incrémenté. À la fin de la tâche, si Nouvelle Paire est toujours à 0, cela signifie qu'aucune nouvelle arête ne peut être rattachée aux composants existants : le calcul est terminé. Sinon, une nouvelle itération est nécessaire.

Il ne reste plus qu'à calculer le nombre de composants connectés en comptant les identifiants de composant distincts.

VII. Résultats

A. Représentation du traitement par l'algorithme CCF

Pour illustrer l'effet de l'algorithme CCF, une fraction de 500 arêtes a été extraite du jeu de données Google. Avant le traitement, dans les quatre groupes identifiés, plusieurs nœuds présentaient de nombreux liens. Pour relier deux nœuds au sein d'un même groupe, il était parfois nécessaire de passer par plusieurs arêtes (Figure 1A).

Après application de l'algorithme CCF, qui redéfinit pour chaque nœud sa connexion vers le nœud le plus petit du groupe. Obtention d'un nœud principal qui centralise les liens du groupe. Le chemin entre deux nœuds est donc réduit à un maximum de deux arêtes (Figure 1B).

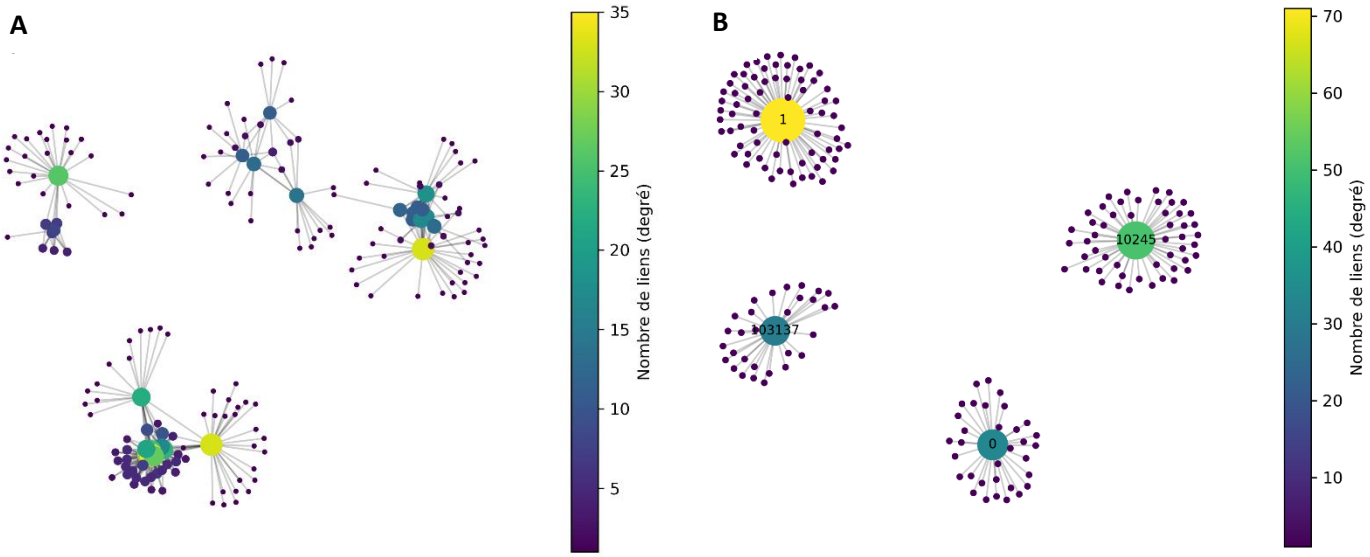


Figure 1 : Représentation des 500 premières arêtes du jeu de données Google. L'échelle de couleur indique le nombre de connexions de chaque nœud.

- **A.** Sans application de l'algorithme CCF.
- **B.** Après application de l'algorithme CCF, avec indication du nœud principal.

B. Représentation du jeu de donnée Google

Pour le jeu de données Google, on dénombre initialement **5 105 039 arêtes**. Après application de l'algorithme CCF, ce nombre est réduit à **872 967 arêtes**, réparties en **2 746 groupes,** pour un total de 875.713 nœuds. Ce résultat est concordant avec les informations du fichier Google, lignes numéro 3 Nodes = 875.713 = nombre de groupes + arêtes post traitement CCF. Le plus grand groupe contient **855 802 nœuds**, tandis que les plus petits ne comptent que **2 nœuds** (Figure 2).

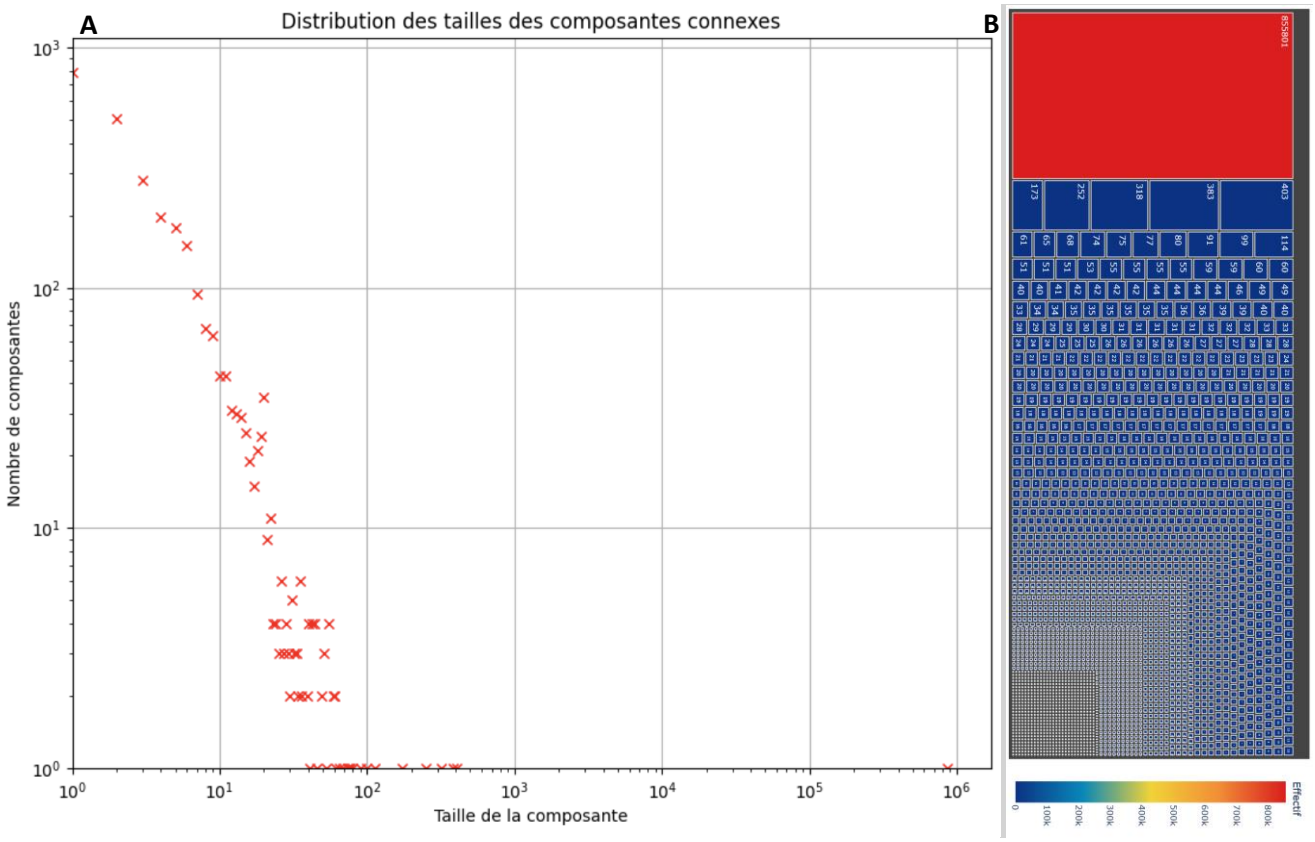


Figure 2 : Représentation des composantes connexes du dataset Google

- **A.** Doptplot, représentation du nombre de groupe en fonction de la taille de la composante connexe.
- **B.** Treemap, représente le nombre de nœuds dans chaque composante connexe du jeu de données Google. Le groupe principal en rouge est tellement disproportionné que sa taille a été réduite sur le graphique.

Pour le jeu de donnée NotreDame, il n'est pas pertinent de faire de tels représentations graphiques, car il n'y a qu'une seule composante connexe.

C. Traitement en Python

1. Machine 1 sans paramètres Spark : local[*]

Une première analyse a été réalisée, avec le **pipeline 1**, dans la **machine 1** en configurat Spark pour utiliser tous les cœurs disponibles, sans ajustement d'autres paramètres. La comparaison a été réalisée entre l'exécution en Python seul et l'utilisation de Spark (DataFrame et RDD, parallélisation fixée à 1 et 6) avec l'algorithme CCF-Iterate + CCF-Dedup.

Jusque 10.000 arrêtes, les trois approches sont équivalentes en terme de performance. Au-delà, nous pouvons observer que l'analyse en Python avec pandas augmente drastiquement, tandis que l'utilisation de PySpark reste plus stable (Figure 3A).

Pour le traitement en RDD, la parallélisation a été fixées à 1 ou 6. Pour les DataFrames, la parallélisation est normalement gérée automatiquement. Cependant, même en tentant de forcer le partitionnement via la fonction coalesce (et non repartition, afin de limiter le *shuffle*), l'étape de *shuffle and sort* renvoyait une seule partition, ce qui explique les performances moins bonnes obtenues avec les DataFrames. (Figure 3B)

En RDD, l'absence de parallélisation augmente le temps d'exécution d'un facteur 3 par rapport à une parallélisation à 6. Ce comportement est lié à la configuration matérielle de la machine de test, qui dispose uniquement de 3 CPU. (Figure 3B)

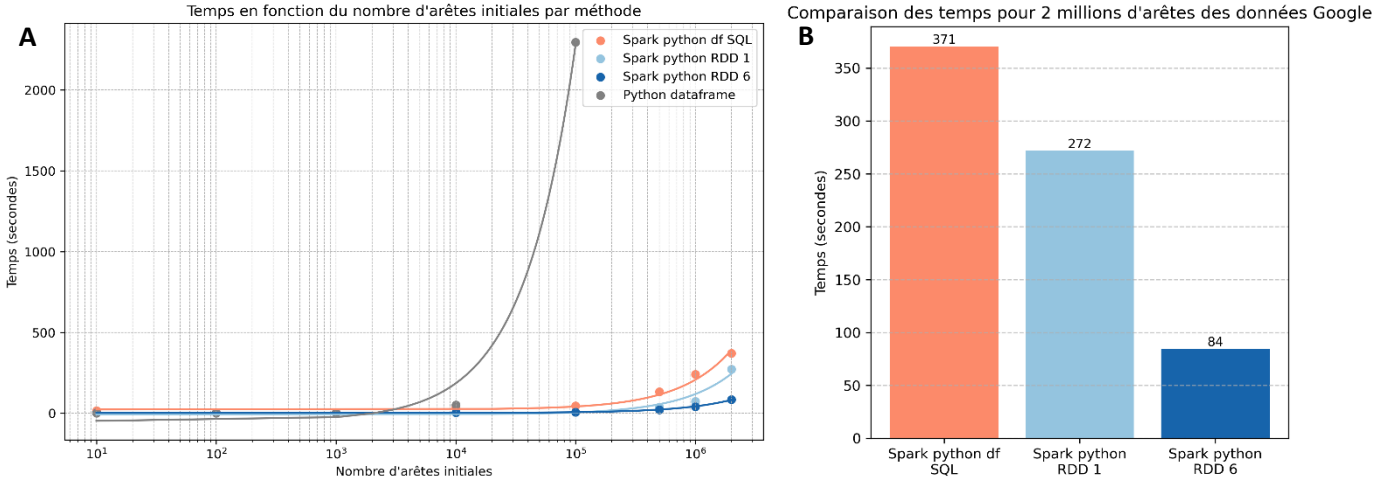


Figure 3 : Analyse des performances en Python avec l'algorithme **CCF-Iterate + CCF-Dedup** (itération avec suppression des doublons).

- **A :** traitement avec ≤ 2 millions d'arêtes
- **B :** traitement avec 2 millions d'arêtes

À noter que dans ce contexte, l'analyse complète du jeu de données Google en RDD (parallélisation 1 ou 6) ou en DataFrame n'a pas pu être terminée du fait d'un problème mémoire. Aucun test intermédiaire n'a été effectué, mais avec une parallélisation de 50 sur le RDD, l'algorithme parvient à se terminer, bien que le temps d'exécution soit long (12 minutes), probablement en raison d'une parallélisation excessive dépassant les capacités réelles de la machine.

Sur la base de ces résultats préliminaires, les paramètres du SparkContext ont été modifiés et des configurations mémoire et performance ont été ajoutées selon la Configuration_1 (voir section Matériel et Méthode). De plus, la variation des performances selon le nombre de partitions a motivé l'étude spécifique de l'effet de la parallélisation.

2. Effet de la parallélisation

Pour étudier l'effet de la parallélisation sur l'algorithme CCF, le même jeu de données Google a été traité en modifiant uniquement le nombre de partitions parallèles. On observe d'abord une diminution du temps de traitement lorsque le degré de parallélisation augmente. Cependant, au-delà d'un certain seuil, le temps de traitement recommence à augmenter (Figure 4).

Cette tendance est liée aux ressources limitées de la machine utilisée (3 CPU). Pour un petit jeu de données (100 000 arêtes), la parallélisation optimale est donc de 3 partitions, chaque CPU traitant une fraction des données simultanément. Lorsque le nombre de partitions dépasse le nombre de CPU, certaines partitions doivent attendre leur tour et l'augmentation du nombre de partition implique un coût supplémentaire lors du regroupement, ce qui entraîne un allongement du temps total.

En revanche, pour des volumes de données plus importants (5 millions d'arêtes), il est possible d'augmenter le nombre de partitions afin de réduire la taille de chaque partition et d'éviter de saturer la mémoire RAM. Spark optimise le temps d'exécution en chargeant les données en mémoire, et des partitions trop volumineuses peuvent provoquer des problèmes de mémoire.

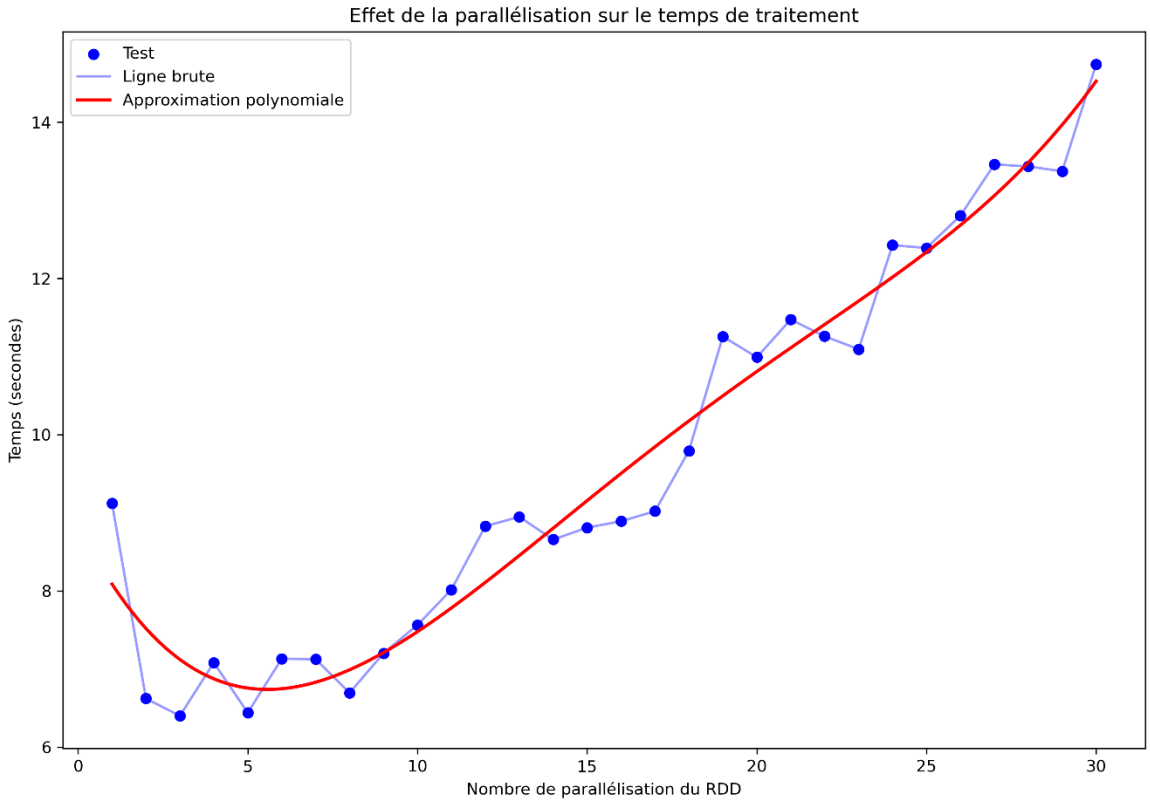


Figure 4 : Traitement des 100 000 premières arêtes du jeu de données Google avec l'algorithme **CCF-Iterate + CCF-Dedup = Standard**. Analyse du temps moyen de traitement (n=4) en fonction du nombre de partitions parallèles du RDD.

3. Machine 2

Le traitement des jeux de données NotreDame et Google a été réalisé avec le **pipeline 2** en Python, en utilisant DataFrame et RDD pour comparer CCF-Iterate + CCF-Dedup = standard. Vs CCF-Iterate sort = secondaire sur la machine 2 avec la configuration 2 (voir matériel et méthode).

On constate que la version DataFrame est systématiquement plus performante que la version RDD. En effet, l'algorithme CCF s'exécute environ **3 fois plus vite** sur DataFrame, quelle que soit la taille du dataset (Figure 5).

Comme l'expliquent les auteurs, l'amélioration du « tri secondaire » n'est pertinente que pour les graphes relativement grands. Dans ce projet, nous utilisons les graphe web de Google et Notredame qui sont probablement trop petit pour observer un gain de temps.

Comparaison des performances dataset NotreDame

Nb of connected components: 1

Nb of edges : **1 497 134**

Classement (du plus rapide au plus lent):
---
1. DF_SECONDAIRE : 168.98s (2.82min)
2. DF_STANDARD : 170.10s (2.84min)
3. RDD_SECONDAIRE : 526.40s (8.77min)
4. RDD_STANDARD : 574.47s (9.57min)

Meilleure méthode: df_secondaire (168.98s)
Ratio d'amélioration: 3.40x plus rapide
Économie de temps: 405.50s

Comparaison des performances dataset Google

Nb of connected components: 2 746

Nb of edges : **5 105 039**

Classement (du plus rapide au plus lent):
---
1. DF_STANDARD : 528.79s (8.81min)
2. DF_SECONDAIRE : 565.39s (9.42min)
3. RDD_STANDARD : 1673.56s (27.89min)
4. RDD_SECONDAIRE : 1675.03s (27.92min)

Meilleure méthode: df_standard (528.79s)
Ratio d'amélioration: 3.17x plus rapide
Économie de temps: 1146.24s

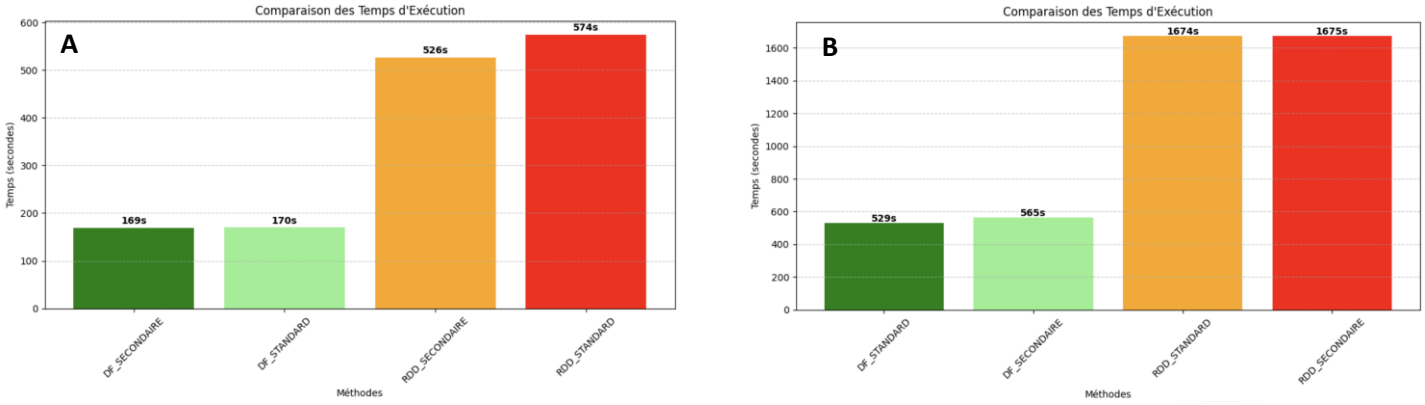


Figure 5 : Analyse des performances en Python sur dataframe et RDD

- **A :** traitement du dataset NotreDame 1,5 millions d'arêtes
- **B :** traitement du dataset Google, 5 millions d'arêtes

4. Comparaison machine et pipeline

Deux machines aux configurations différentes et deux pipelines Python ont été utilisés pour réaliser des tests sur le dataset Google, avec différentes combinaisons.

- Dans tous les tests, la version DataFrame s'est révélée systématiquement plus performante que la version RDD
- L'exécution d'un même pipeline sur deux machines différentes, avec des configurations distinctes, ne produit pas les mêmes temps d'exécution (Figures 6A et 6B).
- De même, l'utilisation de différents pipelines sur une même machine, avec les mêmes paramètres, impacte également le temps d'exécution (Figures 6B et 6C).

Ainsi, bien que les DataFrames offrent les meilleures performances globales, celles-ci restent dépendantes à la fois de la configuration matérielle et de l'implémentation du code Python.

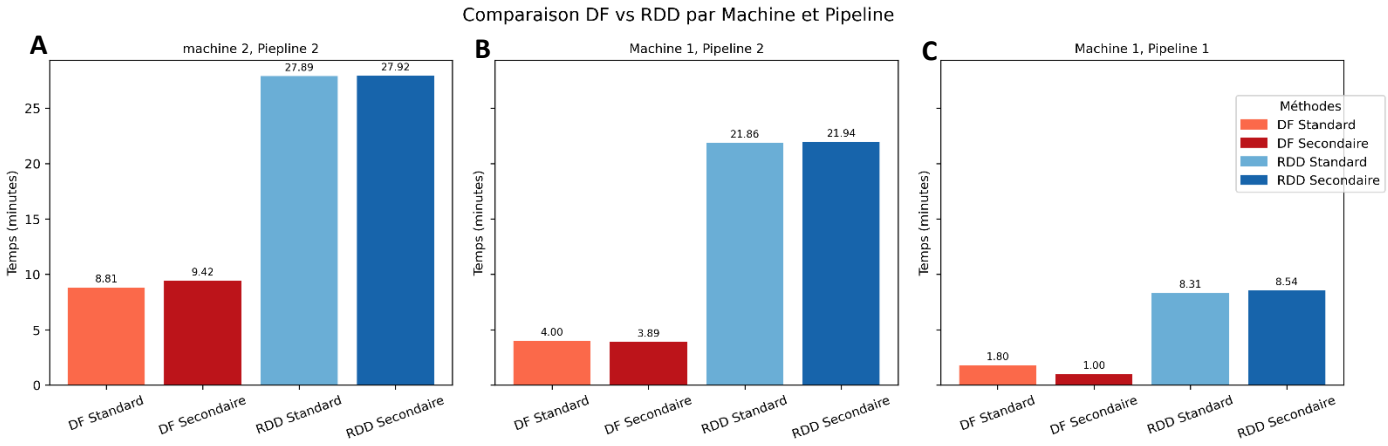


Figure 6 : Analyse des performances en Python sur dataframe et RDD du jeu de donnée Google

- **A :** traitement avec le pipeline 2 sur la machine 2 (16Go RAM CPU 8 cœurs, GPU 8 cœurs + Spark config_2)
- **B :** traitement avec le pipeline 2 sur la machine 1 (8Go RAM et 4 cœurs, GPU 2 cœurs + Spark config_1)
- **C :** traitement avec le pipeline 1 sur la machine 1 (8Go RAM et 4 cœurs, GPU 2 cœurs + Spark config_1)

5. Conclusion Python

L'application de CCF-Iterate sur le graphe web Google permet de retrouver efficacement toutes les composantes connexes. Les deux pipelines développés exploitent pleinement le paradigme MapReduce.

Dans ce projet, nous avons implémenté les deux versions de l'algorithme (RDD et DataFrame) et démontré leur capacité à traiter un graphe même sur des machines aux ressources limitées.

L'utilisation des DataFrames demeure la solution la plus performante, tout en restant sensible à la configuration matérielle et l'implémentation du code Python.

D. Scala

Comme mentionné précédemment, la parallélisation optimale est fixée à 3 pour la machine 1, ce qui a été retenu pour le traitement en RDD. Pour un faible volume de données (< 10 000 arêtes), le traitement est plus rapide avec les **DataFrames** qu'avec les **RDD** (Figure 7A).

Lorsque le volume de données augmente, l'algorithme **CCF-Iterate seul** devient moins performant si les doublons ne sont pas supprimés au fil des itérations. Avec 100 000 arêtes, le temps de calcul est divisé par deux en DataFrame et par trois en RDD lorsque les doublons sont supprimés. Ce qui confirme la conclusion de l'article de référence, l'élimination des paires dupliquées permet de réduire significativement le temps d'exécution, aussi bien avec les DataFrames qu'avec les RDD, et cet effet devient particulièrement visible lorsque le volume de données augmente (Figure 7A et 7B).

En revanche, les traitements **CCF-Iterate + CCF-Dedup = standard** et **CCF-Iterate-Sort = secondaire** donnent des résultats très similaires, que ce soit en DataFrame ou en RDD (Figure 4A et 4B). Une comparaison effectuée sur l'ensemble du jeu de données (5 millions d'arêtes) confirme également cette similarité des performances entre les deux approches en RDD (Figure 7C).

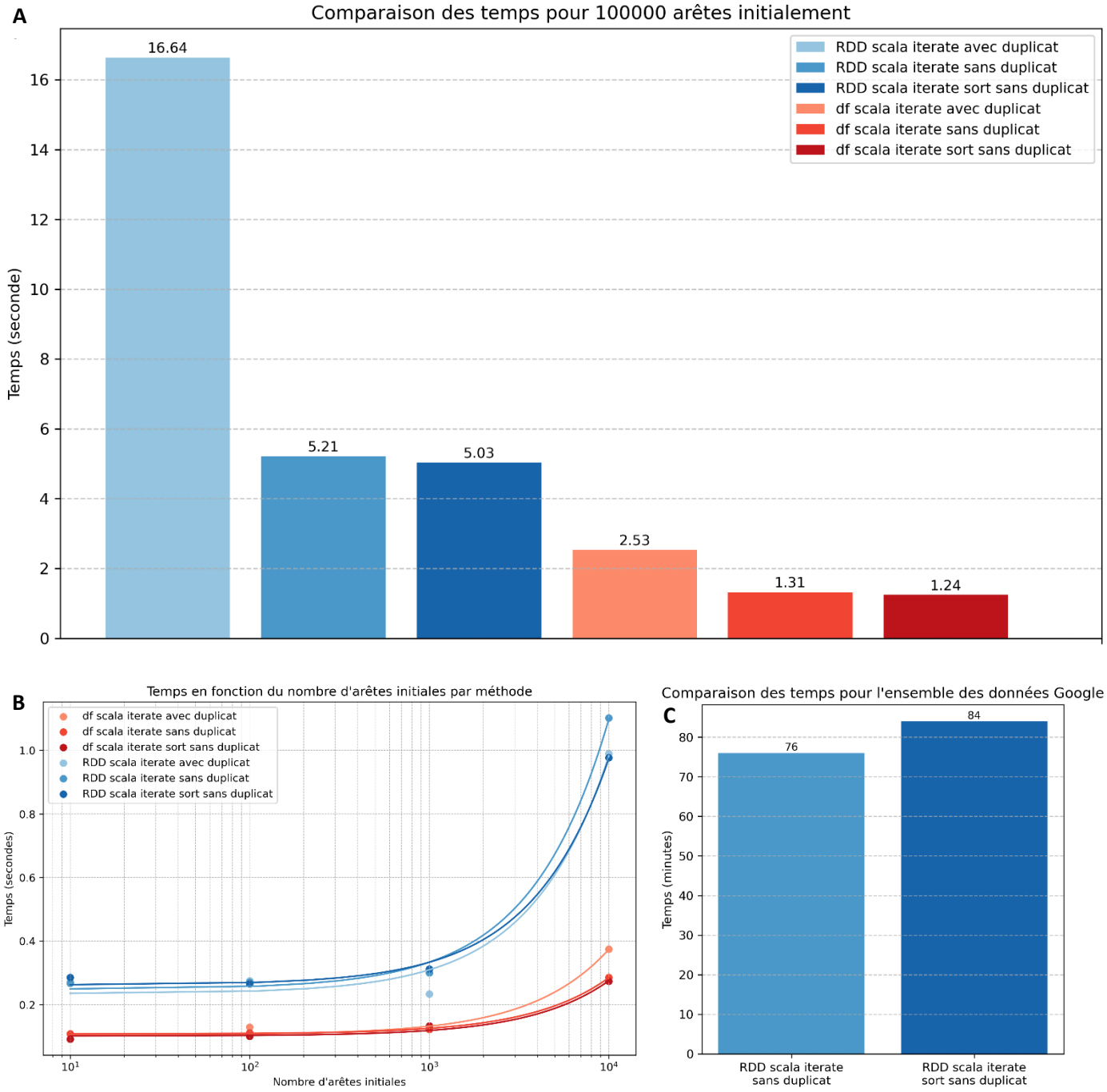


Figure 7 : Analyse des performances en Scala pour trois variantes de l'algorithme :

- **CCF-Iterate** : itération avec doublons
- **CCF-Iterate + CCF-Dedup = standard** : itération avec suppression des doublons
- **CCF-Iterate-Sort = secondaire** : itération triée avec suppression des doublons

Les tests ont été réalisés avec des structures de données de type **DataFrame** et **RDD**.

- **A :** traitement avec 100 000 arêtes
- **B :** traitement avec ≤ 10 000 arêtes
- **C :** traitement de l'ensemble des données (≈ 5 millions d'arêtes)

E. Comparaison python versus scala

Pour le traitement complet du jeu de donnée Google en RDD sur la machine 1. Avec Python, sans configuration spécifique de Spark et une parallélisation de 50 sur le RDD, le temps d'exécution est de 12 minutes (non montré), tandis qu'avec la Configuration_1 et une parallélisation à 3, il est de 8 minutes 30 (Figure 6C). En comparaison, en Scala, sur des conditions équivalentes, le temps d'exécution atteint environ 80 minutes (Figure 7C).

Dans ce test, Python a donc donné de meilleurs résultats.

VIII. Conclusion

Dans ce projet, l'algorithme **Connected Component Finder (CCF)** a été implémenté et testé en Scala et Python, en utilisant à la fois les structures RDD et DataFrame dans Spark. L'objectif était de mettre en pratique le traitement distribué et d'évaluer l'impact de la parallélisation et du choix de la structure de données sur les performances de l'algorithme.

La mise en place de l'environnement de test avec Scala a présenté des difficultés, et les limitations matérielles ont dû être gérées du mieux possible par des personnes encore relativement novices dans l'écosystème Spark. Malgré ces contraintes, les résultats ont clairement montré

- l'intérêt de Spark pour le traitement de données massives
- l'importance de la parallélisation et de la configuration
- l'efficacité des **DataFrames**, plus performantes que les RDD

Au vu de nos observations, l'utilisation de **Spark** en **Python** avec **DataFrame** et une parallélisation adaptée semble la solution la plus efficace.

En conclusion, ce projet illustre l'efficacité de l'algorithme CCF pour le traitement de graphes massifs et souligne l'importance de la parallélisation et de la gestion des doublons pour optimiser les performances dans un contexte de traitement distribué. Il est probable que des optimisations supplémentaires, tant au niveau du code que de l'environnement Spark (gestion de la mémoire, partitionnement, shuffle, etc.), pourraient améliorer les performances, tant en Python qu'en Scala.